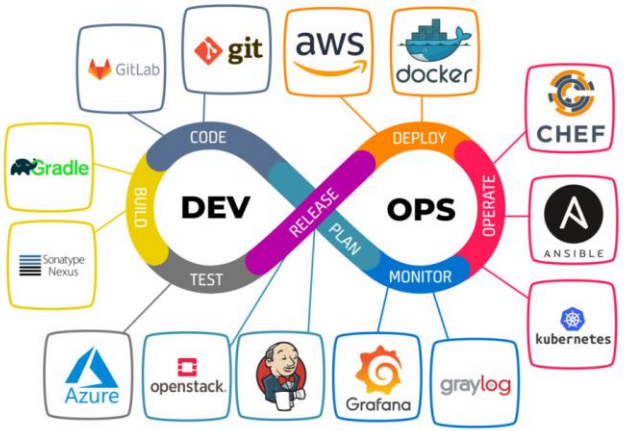
TEK-UP
Ecole Supérieure Privée
Technologie & Ingénierie



Gestion des versions décentralisées

CHAPITRE 2

OPÉRATIONS SUR GIT : ROLLBACK

Sur Git

Modifier un dépôt Git

- Ajouter ou modifier des fichiers dans un projet et actualiser un dépôt Git
- Supprimer un fichier d'un projet et ou l'exclure du suivi de version Git
- Retirer un fichier de la zone d'index de Git
- Empêcher l'indexation de certains fichiers dans Git
- Renommer un fichier dans Git

Annuler des actions et consulter l'historique

- Consulter l'historique des modifications Git
- Annuler des modifications apportées à un fichier

Quelques commandes utiles

- git reflog ➔ affiche les actions actions réalisées en local (eq. git log)
- git blame ➔ permet de savoir qui a réalisé les modifications (date, …)

Rollback

- Comment réinitialiser (reset) un commit Git ?
- Comment annuler (revert) un commit Git ?
- Rétablir ou réinitialiser (Revert or reset) un commit Git ?

Modifier un dépôt Git

Ajouter ou modifier des fichiers dans un projet et actualiser notre dépôt Git

- A chaque fois qu'on souhaite enregistrer une modification de fichier ou un ajout de fichier dans le dépôt Git, nous utilisons les commandes git add puis git commit.
- Le **commit** (la validation / l'enregistrement en base de données) d'un fichier se basera sur **l'état de ce fichier** au moment du git add.
- Cela signifie que si nous effectuons une commande git add sur un fichier puis **modifier** à nouveau le fichier puis effectuer un git commit, c'est le fichier dans son état au moment du **dernier** git add qui sera **validé** et les **dernières modifications** ne seront donc pas enregistrées dans le dépôt Git.
- Si nous souhaitons enregistrer toujours la dernière version d'un fichier, on doit donc toujours **effectuer** un git add juste avant un git commit.
- Si nous souhaitons faire des modifications dans un commit déjà enregistré dans le dépôt (commit existant), on utilise git commit --amend.

Modifier un dépôt Git

Supprimer un fichier d'un projet et / ou l'exclure du suivi de version Git

- Supprimer un fichier de dossier avec la commande Bash rm **ne suffira pas** pour que Git **oublie ce fichier** : il apparaitra dans chaque git status dans une section "changes not stages for commit" (modifications qui ne seront pas validées).

Exemple : Ajouter un fichier test.txt dans un dossier projet et lancer git status.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test.txt

nothing added to commit but untracked files present (use "git add" to track)
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add test.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m "Ajout du fichier test.txt"
[master ebaa3b9] Ajout du fichier test.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test.txt
```

Modifier un dépôt Git

Supprimer un fichier d'un projet et / ou l'exclure du suivi de version Git

- Supprimer avec la commande rm.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ rm test.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    test.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Comme vous pouvez le voir, Git continue de suivre le fichier et la suppression simple du fichier ne sera pas validée comme changement par Git.

Modifier un dépôt Git

Supprimer un fichier d'un projet et / ou l'exclure du suivi de version Git

- Pour supprimer un fichier et l'exclure du suivi de version, nous allons utiliser la commande git rm (et non pas simplement une commande Bash rm).

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git rm test.txt #supprime le fichier du projet et du suivi de version
rm 'test.txt'

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    test.txt


TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m "Suppression de test.txt du projet et du suivi"
[master b8f666c] Suppression de test.txt du projet et du suivi
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 test.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Modifier un dépôt Git

Supprimer un fichier d'un projet et / ou l'exclure du suivi de version Git

- Pour simplement exclure un fichier du suivi Git mais le conserver dans le projet, on va utiliser la même commande git rm mais avec cette fois-ci une option --cached.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ touch fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m 'Ajout du fichier fichier2.txt'
[master dac6945] Ajout du fichier fichier2.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 fichier2.txt
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git rm --cached fichier2.txt
rm 'fichier2.txt'
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    fichier2.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        fichier2.txt
```

Modifier un dépôt Git

Supprimer un fichier d'un projet et / ou l'exclure du suivi de version Git

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m 'Abandon du suivi de fichier2'
[master ffe4962] Abandon du suivi de fichier2
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 fichier2.txt
```

- Ici, le fichier a bien été **exclu** du suivi Git, mais il existe toujours dans notre projet. On va ensuite pouvoir modifier ce fichier (lui ajouter du texte par exemple) comme n'importe quel fichier et Git ne va pas prendra en considération ces modifications.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ echo 'Ajout du texte' >> fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ cat fichier2.txt
Ajout du texte

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status #pour vérifier que git ne suit plus le fichier2.txt
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        fichier2.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Modifier un dépôt Git

Retirer un fichier de la zone d'index de Git

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add fichier2.txt
warning: in the working copy of 'fichier2.txt', LF will be replaced by CRLF the next ti
me Git touches it

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m 'Sauvegarde fichier2.txt'
[master 121a1f9] Sauvegarde fichier2.txt
 1 file changed, 1 insertion(+)
 create mode 100644 fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ echo 'Modification du fichier 2'>> fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ cat fichier2.txt
Ajout du texte
Modification du fichier 2

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add fichier2.txt
warning: in the working copy of 'fichier2.txt', LF will be replaced by CRLF the next ti
me Git touches it

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   fichier2.txt
```

Ici, on ajoute à nouveau le fichier fichier2.txt à l'index et on le passe donc sous suivi de version. On valide cela avec git commit puis on modifie le contenu de notre fichier et on ajoute la dernière version de notre fichier à la zone d'index pour qu'il fasse partie du prochain commit.

Modifier un dépôt Git

Retirer un fichier de la zone d'index de Git

- Le contenu de la zone d'index est ce qui sera proposé lors du prochain commit.
- Imaginons qu'on a ajouté un fichier à la zone d'index par erreur.
- Pour retirer un fichier de l'index, on peut utiliser git reset HEAD nom-du-fichier.
- A la différence de git rm, le fichier continuera d'être suivi par Git. Seulement, le fichier dans sa version actuelle va être retiré de l'index et ne fera donc pas partie du prochain commit.

HEAD : un pointeur, une référence sur votre position actuelle dans votre répertoire de travail Git.
master : une référence qui pointe vers la tête de l'historique.

Modifier un dépôt Git

Retirer un fichier de la zone d'index de Git

- Finalement, on change d'idée et on veut retirer cette version de la zone d'index.
- On utilise git reset HEAD fichier2.txt.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git reset HEAD fichier2.txt #retirer le fichier de la zone d index
Unstaged changes after reset:
M       fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   fichier2.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Modifier un dépôt Git

Empêcher l'indexation de certains fichiers dans Git

- Lorsqu'on dispose d'un projet et qu'on souhaite utiliser Git pour effectuer un suivi de version, il est courant qu'on souhaite exclure certains fichiers du suivi de version : comme les fichiers générés automatiquement, les fichiers de configuration, les fichiers sensibles, etc.

- On peut informer Git sur ces fichiers qu'on ne souhaite pas indexer en créant un fichier .gitignore et en ajoutant les différents fichiers qu'on souhaite ignorer.
- On peut également mentionner des schémas de noms pour exclure tous les fichiers correspondant à ce schéma et qu'on peut même exclure le contenu entier d'un répertoire en écrivant le chemin du répertoire suivi d'un slash.

Modifier un dépôt Git

Empêcher l'indexation de certains fichiers dans Git

1. Créer un fichier .gitignore :

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ touch .gitignore
```

2. Configurer votre projet

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git config --global core.excludesfile ~/.gitignore_global
```

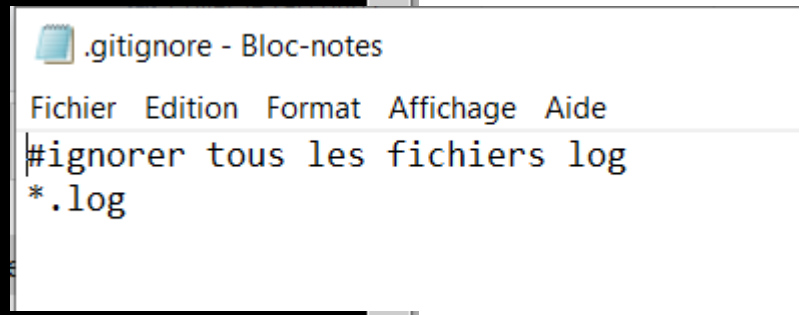
3. Editer et tester le fichier .gitignore:

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add .gitignore
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        modified:   fichier2.txt


TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ touch test.log

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        modified:   fichier2.txt
```



Modifier un dépôt Git

Renommer un fichier dans Git

- On peut également renommer un fichier de notre projet depuis Git en utilisant cette fois-ci une commande git mv ancien-nom-fichier nouveau-nom-fichier.

- On peut par exemple renommer notre fichier “readme.txt” en “lisezmoi.txt” de la manière suivante :

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ ls
README.txt  fichier1.txt  fichier2.txt  test.log  test2.log  test3.log

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git mv README.txt LISEZMOI.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ ls
LISEZMOI.txt  fichier1.txt  fichier2.txt  test.log  test2.log  test3.log
```

Modifier un dépôt Git

Renommer un fichier dans Git

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    README.txt -> LISEZMOI.txt
```

- Le fichier a bien été renommé dans notre répertoire et le changement est prêt à être validé dans le prochain commit

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -a -m "renommage du fichie"
[master 3a78e39] renommage du fichie
 1 file changed, 0 insertions(+), 0 deletions(-)
 rename README.txt => LISEZMOI.txt (100%)

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Annuler des actions et consulter l'historique

Consulter l'historique des modifications Git

- La commande git log affiche la liste des commits réalisés du plus récent au plus ancien. Par défaut, chaque commit est affiché avec sa somme de contrôle SHA-1, le nom et l'e-mail de l'auteur, la date et le message du commit.

- La commande git log supporte également de nombreuses options.

- Certaines vont pouvoir être très utiles comme par exemple les options :

  -p, --pretty, --since ou --author.

SHA-1 : Secure Hash Algorithm

Annuler des actions et consulter l'historique

Annuler des modifications apportées à un fichier

- L'un des principaux intérêts d'utiliser un logiciel de gestion de vision est de pouvoir travailler en "roll back", c'est-à-dire de pouvoir revenir à un état antérieur enregistré d'un projet.

- Après avoir utilisé un commit, nous pouvons continuer à travailler sur nos fichiers et à les modifier.

➔ Parfois, certaines modifications ne vont pas apporter les comportements espérés et on voudra revenir à l'état du fichier du dernier instantané Git (c'est-à-dire au dernier état enregistré).
➔ On va pouvoir faire ceci avec la commande générale :

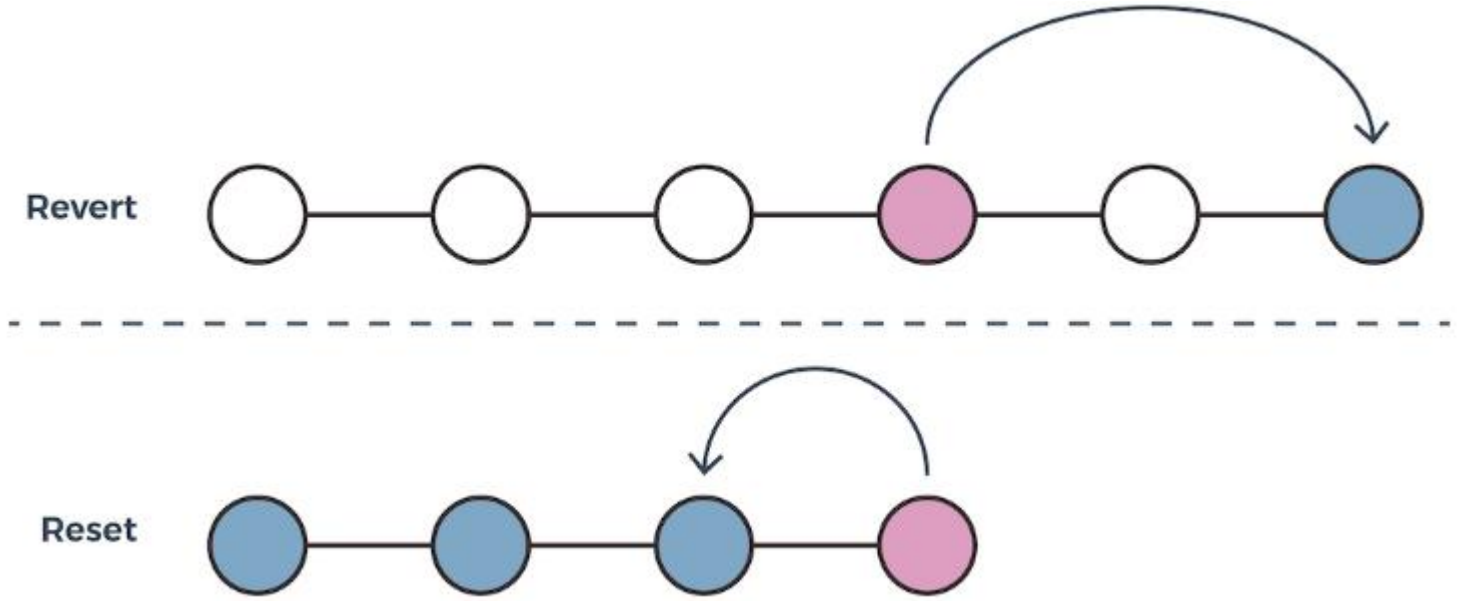
git checkout -- nom-du-fichier ou la nouvelle commande spécialisée git restore nom-du-fichier.

Rollback

Réinitialiser (reset) et annuler (revert) un commit Git ?

- Pour annuler les modifications apportées sur un dépôt, on peut le faire sur git en utilisant deux commandes git «reset» et «revert »

  ✓ reset : permet de revenir à l'état précédent sans créer un nouveau commit.
  ✓ revert : permet de revenir à l'état précédent en créant un nouveau commit.



Rollback

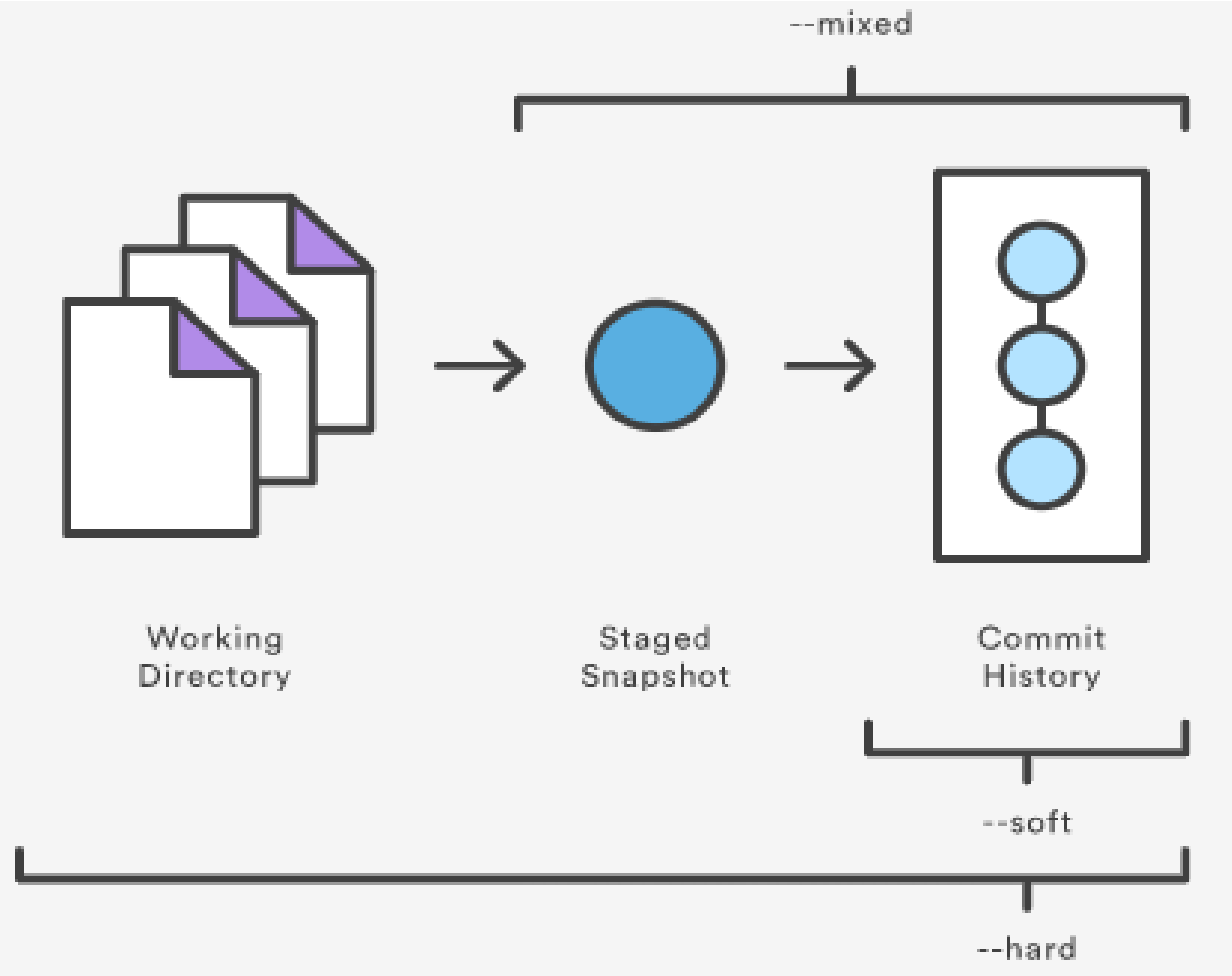
Comment réinitialiser (reset) un commit Git ?

- git reset peut être appelée de trois façons différentes, correspondant aux arguments de ligne de commande

  --soft, --mixed et --hard.

➔ Ces trois arguments correspondent chacun aux trois mécanismes de gestion des états internes de Git, l'arborescence des commits (HEAD), la zone d'index et le répertoire de travail.

➔ Il s'agit de structures de données basées sur des nœuds et des pointeurs que Git utilise pour suivre une chronologie des éditions (version).



Rollback

Comment réinitialiser (reset) un commit Git ?

- La meilleure façon de démontrer ces mécanismes est de créer un ensemble de changements dans un dépôt et de le suivre via les trois arborescences.

➔ Créer un nouveau dépôt Git avec un seul fichier vide : reset_file.
À ce stade, l'exemple de dépôt comporte un seul commit (9fbb710) suite à l'ajout du fichier reset_file.

```
user@DESKTOP-RRVCC4I MINGW64 ~
$ mkdir git_reset

user@DESKTOP-RRVCC4I MINGW64 ~
$ cd git_reset/

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset
$ git init
Initialized empty Git repository in C:/Users/user/git_reset/.git/

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ touch reset_file.txt

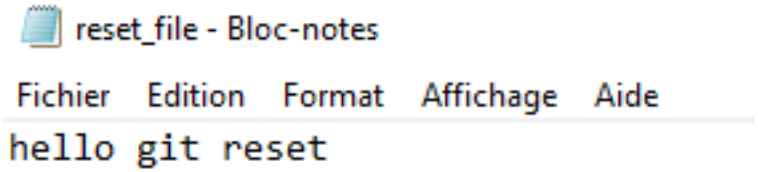
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git add reset_file.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git commit -m "initial commit"
[master (root-commit) 9fbb710] initial commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 reset_file.txt
```

Rollback

Comment réinitialiser (reset) un commit Git ?
Répertoire de travail

➔ Dans le dépôt de démonstration : nous modifions le fichier reset_file et y ajoutons du contenu.



➔ git status montre que Git est conscient des changements apportés au fichier.
Ces changements font actuellement partie de la première arborescence : «répertoire de travail».
La commande git status peut être utilisée pour afficher les changements apportés au répertoire de travail. Ils seront affichés en rouge avec un préfixe 'modified'.

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ echo 'hello git reset' > reset_file.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   reset_file.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        reset_file

no changes added to commit (use "git add" and/or "git commit -a")
```

Rollback

➔ Cette arborescence procède au suivi des changements du répertoire de travail associés à l'action git add, à stocker dans le commit suivant.

- Cette arborescence constitue un mécanisme de mise en cache interne complexe.
- Git tente généralement de masquer à l'utilisateur les détails de mise en œuvre de la zone d'index.
- Afin d'afficher l'état de l'index, nous devons utiliser une commande Git : git ls-files.
- git ls-files -s affiche des métadonnées supplémentaires pour les fichiers dans la zone d'index.
- Il s'agit d'une empreinte SHA-1 d'objet Git standard.
- C'est une empreinte d'un contenu des fichiers.
- L'historique des commits stocke ses propres empreintes SHA d'objet pour identifier des pointeurs vers les commits et les réfs, et l'index dispose de ses propres empreintes SHA d'objet pour suivre les versions des fichiers dans l'index.

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git ls-files -s
100644 e69de29bb2d1d6434b8b29ae775ad8c2e48c5391 0       reset_file.txt
```

Rollback

Comment réinitialiser (reset) un commit Git ?
Zone d'Index

➔ Passer le fichier reset_file modifié dans la zone d'index.

➔Ajoutant le fichier à la zone d'index.

Après git status, reset_file s'affiche en vert sous « Changes to be committed ».

➔ Il convient de noter que git status n'est pas une vraie représentation de la zone d'index.

➔ La sortie de la commande git status affiche les **changements** entre **l'historique des commits** et **l'index de staging**.

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git add reset_file
warning: in the working copy of 'reset_file', LF will be replaced by CRLF the ne
xt time Git touches it

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   reset_file

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   reset_file.txt
```

Rollback

Examinons le contenu de la zone d'index.

➔ Nous pouvons voir que l'objet SHA de reset_file a été mis à jour

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git ls-files -s
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file
100644 e69de29bb2d1d6434b8b29ae775ad8c2e48c5391 0       reset_file.txt
```

Rollback

Comment réinitialiser (reset) un commit Git ?
Historique des commit

La commande git commit ajoute des changements à un instantané permanent dans l'historique des commits. Cet instantané contient également l'état de l'index au moment du commit.

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git commit -am"update content of reset_file"
warning: in the working copy of 'reset_file.txt', LF will be replaced by CRLF the next time Git touches it
[master cc568b3] update content of reset_file
 2 files changed, 2 insertions(+)
 create mode 100644 reset_file

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Nous avons créé un nouveau commit avec un message "update content of reset_file".
L'ensemble de changements a été ajouté à l'historique des commits.
git status montre qu'il n'y a pas de changements en attente dans les arborescences.
L'exécution de git log affichera l'historique des commits.
➔ Nous pouvons commencer à utiliser git reset.

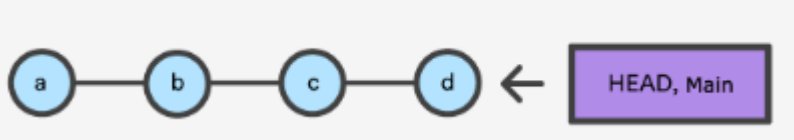
Rollback

Fonctionnement

- git reset a un comportement semblable à git checkout

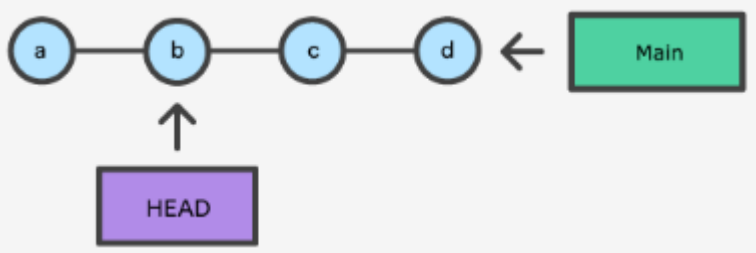
  git checkout agit uniquement sur le pointeur de la réf HEAD

  git reset déplace à la fois le pointeur de la réf HEAD et le pointeur de la réf de la branche courante.

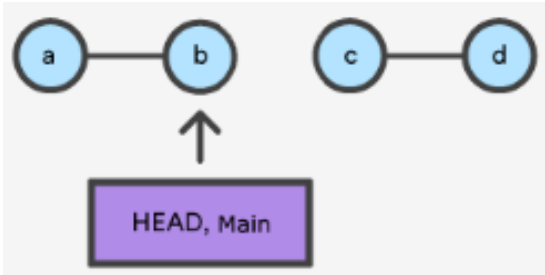


➔Une séquence de commits sur la branche principale Main.

➔ Les réfs HEAD et de branche (Main) pointent sur le commit d.



git checkout b



git reset b

Avec git checkout, la réf main pointe toujours vers d.

La réf HEAD a été déplacée et pointe vers le commit b. Le dépôt est à l'état HEAD détaché.

git reset déplace la réf HEAD et les réfs de branche (Main) vers le commit spécifié.

Rollback

git reset --hard

- Quand on utilise l'argument --hard les pointeurs de la réf d'historique des commits sont mis à jour au commit spécifié.
- L'index et le répertoire de travail sont ensuite réinitialisés de sorte qu'ils correspondent à ceux du commit spécifié.
- Les changements précédemment en attente apportés à l'index et au répertoire de travail sont réinitialisés de sorte qu'ils correspondent à l'état de l'arborescence des commits.

➔ Cela signifie que toute tâche en attente qui traînait dans l'index et le répertoire de travail sera perdue.

➔ Cette commande permet de **revenir à n'importe quel commit** mais en oubliant tout ce qu'il s'est passé après.

Rollback

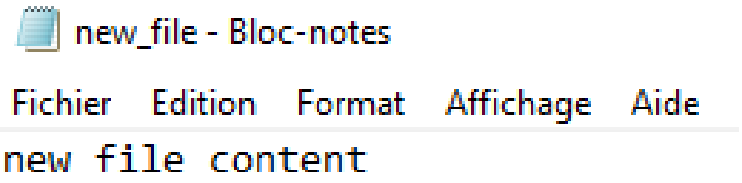
git reset --hard

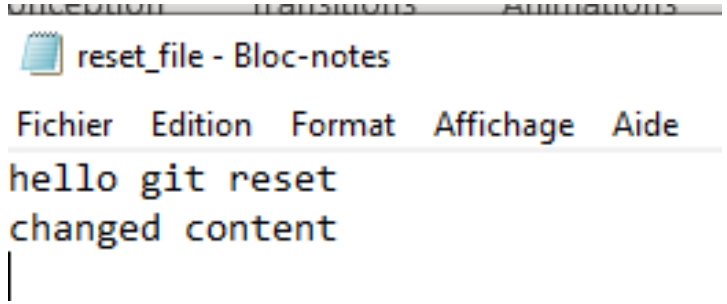
- Apportons des changements au dépôt. Exécutons quelques commandes :

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ echo 'new file content' > new_file.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git add new_file.txt
warning: in the working copy of 'new_file.txt', LF will be replaced by CRLF the next time Git touches it

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ echo 'changed content' >> reset_file.txt
```





- Nous avons créé un nouveau fichier new_file, que nous avons ajouté au dépôt.
- Le contenu du fichier reset_file,txt sera modifié par l'ajout d'une 2ème phrase.

Rollback

git reset --hard

- Une fois ces changements en place, examinons l'état du dépôt à l'aide de git status.

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   new_file
        new file:   new_file.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   reset_file.txt
```

- L'Index de staging présente un changement en attente pour l'ajout de new_file, et le répertoire de travail présente un changement en attente pour les changements apportés à reset_file,txt.

Rollback

git reset --hard

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git ls-files -s
100644 8e66654a5477b1bf4765946147c49509a431f963 0       new_file
100644 8e66654a5477b1bf4765946147c49509a431f963 0       new_file.txt
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file.txt
```

- On remarque que new_file et new_file.txt ont été ajouté à l'index.

- Nous avons effectué des mises à jour sur reset_file.txt, mais l'empreinte SHA de la zone d'index (d7d77c1b04b5edd5acfc85de0b592449e5303770) reste inchangée.

➔ Ce comportement est attendu, car git add n'a pas été utilisée pour promouvoir ces changements dans la zone d'index.
➔ Ces changements apparaissent dans le **répertoire de travail**.

Rollback

git reset --hard

- Exécutons un git reset --hard.

➔ Git affiche une sortie indiquant que HEAD pointe vers le dernier commit cc568b3.

- git status.

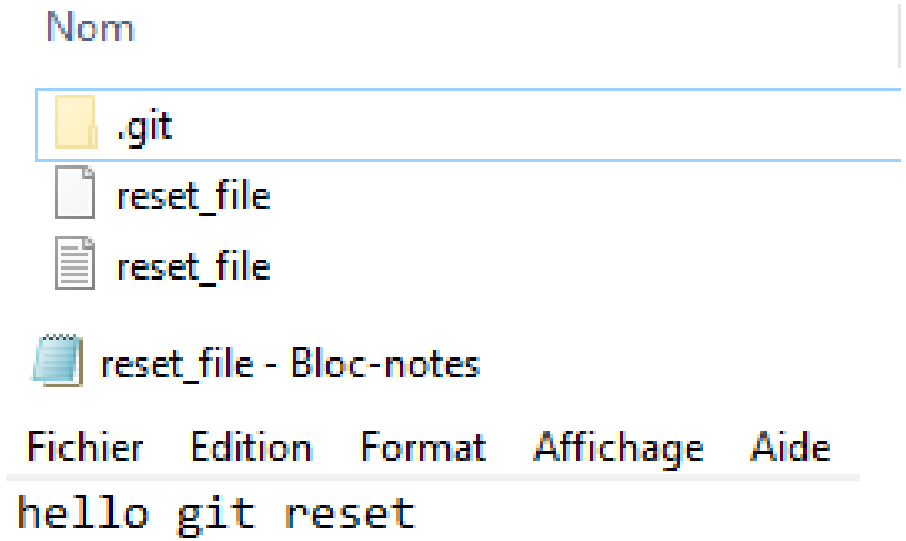
➔ Git indique qu'aucun changement n'est en attente.

➔ Les changements apportés à reset_file, reset_file.txt et l'ajout de new_file et new_file.txt ont été supprimés. Il est essentiel de noter que cette perte de données est définitive.

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git reset --hard
HEAD is now at cc568b3 update content of reset_file

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status
On branch master
nothing to commit, working tree clean

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git ls-files -s
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file.txt
```



Rollback

git reset --mixed

- Il s'agit du mode de fonctionnement par défaut.
- Les pointeurs de réf sont mis à jour.
- L'index est réinitialisé à l'état du commit spécifié.
- Les changements annulés depuis l'index sont déplacés vers le répertoire de travail.

➔ git reset --mixed va permettre de revenir juste après le dernier commit ou le commit spécifié, sans supprimer les modifications en cours.

➔ Il permet aussi, dans le cas de fichiers indexés mais qui n'ont pas encore commités, de désindexer les fichiers.

Rollback

git reset --mixed

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ echo 'new file content' > new_file.txt

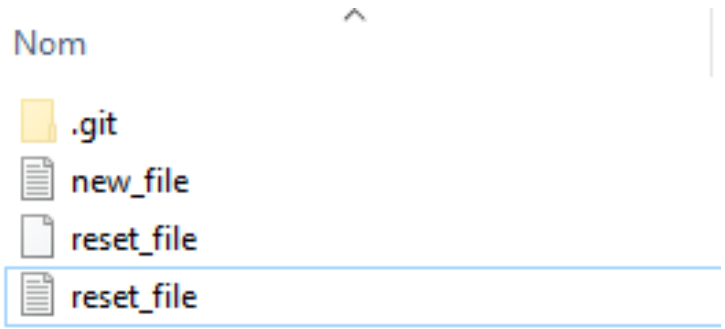
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git add new_file.txt
warning: in the working copy of 'new_file.txt', LF will be replaced by CRLF the next time Git touches it

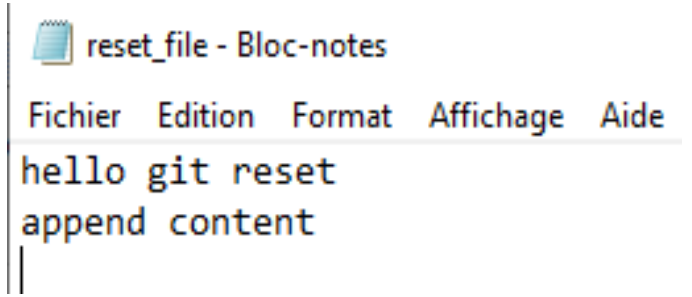
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ echo 'append content' >> reset_file.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git add reset_file.txt
warning: in the working copy of 'reset_file.txt', LF will be replaced by CRLF the next time Git touches it

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   new_file.txt
        modified:   reset_file.txt


user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git ls-files -s
100644 8e66654a5477b1bf4765946147c49509a431f963 0       new_file.txt
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file
100644 0fcccd36ac649422bbe0226d9c0a11d12699f174 0       reset_file.txt
```





Rollback

git reset --mixed

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git reset --mixed
Unstaged changes after reset:
M       reset_file.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   reset_file.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        new_file.txt

no changes added to commit (use "git add" and/or "git commit -a")

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git ls-files -s
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file.txt
```

- git status nous montre que certains changements ont été apportés au fichier reset_file.txt et qu'un fichier non tracké existe : new_file.txt.

➔ Il s'agit du comportement --mixed explicite. L'index a été réinitialisé, et les changements en attente ont été déplacés vers le répertoire de travail.

Rollback

git reset --soft

- git reset -- soft permet de se placer sur un commit spécifique afin de voir le code à un instant donné, ou de créer une branche partant d'un ancien commit.
- Elle **ne supprime aucun fichier**, aucun commit, et ne crée pas de HEAD détaché.
- Lorsque l'argument -- soft est transmis, les pointeurs de réf sont mis à jour et le reset s'arrête.
- L'index et le répertoire de travail ne seront pas modifiés.

Rollback

git reset --soft

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git add reset_file.txt
warning: in the working copy of 'reset_file.txt', LF will be replaced by CRLF the next time Git touches it

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git ls-files -s
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file
100644 0fcccd36ac649422bbe0226d9c0a11d12699f174 0       reset_file.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   reset_file.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        new_file.txt
```

- Utilisant git add pour promouvoir le fichier reset_file.txt modifié dans l'index.
- Nous confirmons que l'index a été mis à jour avec git ls-files -s.
- git status affiche la partie « Changes to be committed » en vert.
- Le fichier new_file.txt mentionné dans les exemples précédents se situe dans le répertoire de travail sous forme de fichier non tracké.

Rollback

git reset --soft

- Nous avons exécuté git reset --soft.
- L'examen de l'état du dépôt avec git status et git ls-files -s montre que rien n'a changé.
- C'est le comportement attendu.

➔ Un soft reset réinitialisera uniquement l'historique des commits.

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git reset --soft

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   reset_file.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        new_file.txt

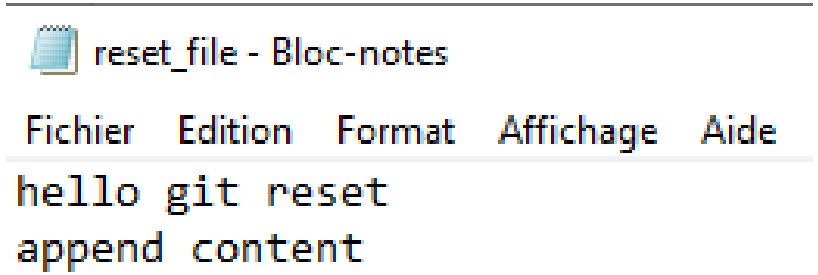

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git ls-files -s
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file
100644 0fcccd36ac649422bbe0226d9c0a11d12699f174 0       reset_file.txt
```

Rollback

git reset --soft

- L'ID de commit qui nous intéresse dans le cadre de cet exemple est `9fbb710329a10102101e3d18875744ccd1a4e78f`.

➔ Cet ID correspond au « commit initial ». Lorsque nous aurons trouvé cet ID, nous l'utiliserons comme cible pour notre soft reset.




```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git commit -m"prepend content to reset_file.txt"
[master f13b41a] prepend content to reset_file.txt
 1 file changed, 1 insertion(+)

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git log
commit f13b41ade7c1b41dbdabde40bd5ea28eb3f4420e (HEAD -> master)
Author: maha-cherif <maha.cherif@gmail.com>
Date:   Mon Apr 17 00:47:54 2023 +0200

    prepend content to reset_file.txt

commit cc568b3ed6808caaa34a241180974c0120c14136
Author: maha-cherif <maha.cherif@gmail.com>
Date:   Sun Apr 16 23:33:32 2023 +0200

    update content of reset_file

commit 9fbb710329a10102101e3d18875744ccd1a4e78f
Author: maha-cherif <maha.cherif@gmail.com>
Date:   Sun Apr 16 22:44:13 2023 +0200

    initial commit
```

Rollback

- Nous exécutons ici les deux commandes git status et git ls-files -s en même temps qui nous indiquent que des changements sont en attente dans le dépôt et que la version du fichier reset_file.txt dans l'index est 0fcccd36ac649422bbe0226d9c0a11d12699f174.
- git status indique que des changements sont apportés à reset_file.txt et les met en évidence en précisant qu'il s'agit de changements pour le commit suivant.
- L'entrée git ls-files -s indique que l'index n'a pas changé et conserve l'empreinte SHA 0fcccd36ac649422bbe0226d9c0a11d12699f174.

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status && git ls-files -s
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        new_file.txt

nothing added to commit but untracked files present (use "git add" to track)
100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file
100644 0fcccd36ac649422bbe0226d9c0a11d12699f174 0       reset_file.txt
```

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git reset --soft 9fbb710329a10102101e3d18875744ccd1a4e78f

user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git status && git ls-files -s
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   reset_file
        modified:   reset_file.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        new_file.txt

100644 d7d77c1b04b5edd5acfc85de0b592449e5303770 0       reset_file
100644 0fcccd36ac649422bbe0226d9c0a11d12699f174 0       reset_file.txt
```

Rollback


```
user@DESKTOP-RRVCC4I MINGW64 ~/git_reset (master)
$ git log 9fbb710329a10102101e3d18875744ccd1a4e78f
commit 9fbb710329a10102101e3d18875744ccd1a4e78f (HEAD -> master)
Author: maha-cherif <maha.cherif@gmail.com>
Date:   Sun Apr 16 22:44:13 2023 +0200

    initial commit
```

- La sortie de git log montre que l'historique des commits ne comprend qu'un seul commit

➔ Cela permet d'illustrer clairement l'effet de l'argument --soft.

- L'argument --soft **ne touche pas l'index**.

➔ C'est pourquoi les mises à jour de notre index sont revenues en arrière dans l'historique des commits.

Rollback

Comment annuler (revert) un commit Git ?

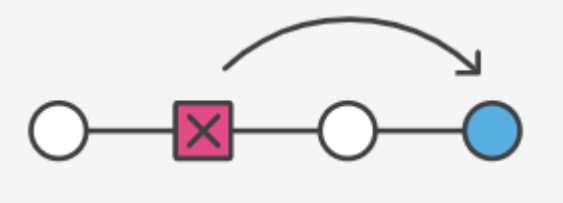
- La commande git revert ne réalise pas une opération d'annulation classique.
- Au lieu de supprimer le commit de l'historique du projet, elle détermine comment annuler les changements introduits par le commit et ajoute un nouveau commit avec le contenu obtenu.

- Ainsi, Git ne perd pas l'historique, lequel est important pour l'intégrité de l'historique de révision.

- L'opération de revert ne doit être utilisée que lorsque nous souhaitons appliquer l'inverse d'un commit à partir de notre historique de projet.
- Cela peut être utile, par exemple, si nous suivons un bug et que nous découvrons qu'il a été introduit par un commit unique.

  Au lieu d'accéder manuellement à un nouvel instantané, de le corriger et de le commiter, nous pouvons utiliser git revert qui le fera automatiquement.

Rollback



Fonctionnement

- La commande git revert est utilisée pour annuler des changements apportés à l'historique de commits d'un dépôt.
- D'autres commandes « d'annulation », comme git checkout et git reset, déplacent les pointeurs de réf HEAD et les branches vers un commit spécifié.
- git revert prend également un commit spécifié.
- Toutefois, git revert ne déplace pas les pointeurs de réf vers ce commit.
- Une opération de revert :

  prend le commit spécifié,
  inverse les changements introduits par le commit,
  crée un « commit de revert ».

➔ Les pointeurs de réf sont ensuite mis à jour pour pointer vers le nouveau commit de revert constituant la pointe de la branche.

Rollback

-initialiser un dépôt dans un répertoire git_revert.

-faire **trois commits** dans le dépôt après avoir ajouté un fichier demo_file et modifié son contenu à deux reprises.

```
user@DESKTOP-RRVCC4I MINGW64 ~
$ mkdir git_revert

user@DESKTOP-RRVCC4I MINGW64 ~
$ cd git_revert/

user@DESKTOP-RRVCC4I MINGW64 ~/git_revert
$ git init
Initialized empty Git repository in C:/Users/user/git_revert/.git/

user@DESKTOP-RRVCC4I MINGW64 ~/git_revert (master)
$ touch demo_file.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git_revert (master)
$ git add demo_file.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git_revert (master)
$ git commit -am "commit initial"
[master (root-commit) 8c7cc71] commit initial
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 demo_file.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git_revert (master)
$ echo "initial content" >> demo_file.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git_revert (master)
$ git commit -am "add new content to demo file"
warning: in the working copy of 'demo_file.txt', LF will be replaced by CRLF the next time Git touches it
[master efaca9a] add new content to demo file
 1 file changed, 1 insertion(+)

user@DESKTOP-RRVCC4I MINGW64 ~/git_revert (master)
$ echo "prepended line content" >> demo_file.txt
```

Rollback

-Faire git log ou bien git log -- oneline pour consulter l'historique du commit, et en affichant un total de trois commits.

-initier git revert.

MINGW64:/c/Users/user/git_revert

```
Revert "prepend content to demo file"

This reverts commit b775a57ee2a806436af5c12157336486c1987e81.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
# Changes to be committed:
#       modified:   demo_file.txt
#
~
```

```
user@DESKTOP-RRVCC4I MINGW64 ~/git_revert (master)
$ git log
commit b775a57ee2a806436af5c12157336486c1987e81 (HEAD -> master)
Author: maha-cherif <maha.cherif@gmail.com>
Date:   Sun Apr 16 16:07:25 2023 +0200

    prepend content to demo file

commit efaca9aac6c5d56681b96412a16a6c5cdc3f1845
Author: maha-cherif <maha.cherif@gmail.com>
Date:   Sun Apr 16 16:05:52 2023 +0200

    add new content to demo file

commit 8c7cc71e72eacd0b6cd94980494add2974467636
Author: maha-cherif <maha.cherif@gmail.com>
Date:   Sun Apr 16 16:03:43 2023 +0200

    commit initial

user@DESKTOP-RRVCC4I MINGW64 ~/git_revert (master)
$ git log --oneline
b775a57 (HEAD -> master) prepend content to demo file
efaca9a add new content to demo file
8c7cc71 commit initial

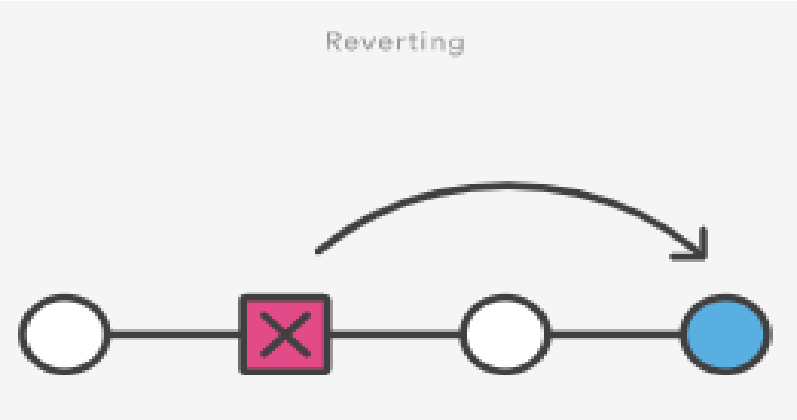
user@DESKTOP-RRVCC4I MINGW64 ~/git_revert (master)
$ git revert HEAD
[master 990243c] Revert "prepend content to demo file"
 1 file changed, 1 deletion(-)

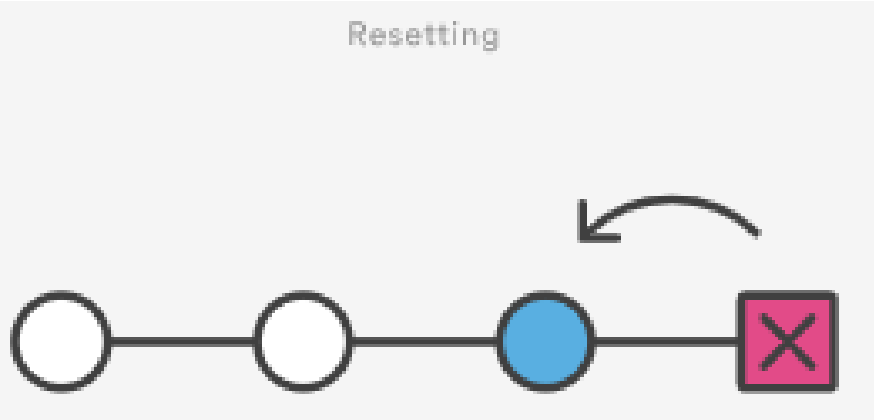
user@DESKTOP-RRVCC4I MINGW64 ~/git_revert (master)
$ git log --oneline
990243c (HEAD -> master) Revert "prepend content to demo file"
b775a57 prepend content to demo file
efaca9a add new content to demo file
8c7cc71 commit initial
```

Rollback

Rétablir ou réinitialiser (Revert or reset) un commit Git ?

- Il est important de comprendre que la commande git revert annule un commit unique.
- Elle ne permet pas de « revenir » à l'état précédent d'un projet en supprimant tous les commits suivants.
- Dans ce cas dans git, cette commande est nommée reset, et non revert.





➔La commande revert présente deux avantages par rapport à la commande reset.

Elle ne change pas l'historique du projet,

Elle peut cibler un commit individuel à un point arbitraire de l'historique, tandis que reset permet seulement de revenir en arrière à partir du commit actuel.